1. Problem Framing and Dataset Analysis

Monthly yellow taxi trip reports are available on the New York City Taxi and Limousine Commission (TLC) website. The dataset for this project was chosen for January 2019. It contains raw records of 7,696,617 trips, including pickup and drop-off dates and times, trip distance, fare amount, number of passengers, and pickup and drop-off location IDs.

The data was prepared using a 143 MB Parquet file containing the trip records, a CSV file listing the taxi zones, and a GeoJSON file with taxi zone polygons. Around 200,000 records were removed because they failed our validation rules. A total of 7,490,668 records passed the checks for exact duplicates, missing values, pickup and drop-off times, distance, fares, passenger count, and valid pickup and drop-off location IDs. The excluded records were saved in data/processed/exclusions.csv for inspection. These rules were based on reasonable assumptions about taxi trips. They included limiting distances to 0–200 miles to identify potential sensor errors, fares to 0−500 to identify possible test or erroneous records, passenger counts to 1–6, checking pickup and drop-off zone IDs against taxi_zones, and checking pickup and drop-off timestamps. During feature engineering, we found that another 774 records had calculated speeds above our maximum of 120 mph. Their timestamps and distances were valid, but their calculated speeds were unusually high, possibly because of incorrect location records. This taught us that calculated speed is an additional feature that needs its own validation rule.

2. System Architecture and Design Decisions

The system comprises five components: data sources, the ETL pipeline, the database, the Flask API, and the browser dashboard.

We picked Flask as our web framework because most of our processing was done in Python or SQL, and it is a lightweight framework. We chose PostgreSQL over SQLite because proper indexing could help with performance when working with a large dataset. To give us more control over database queries and protect against SQL injection, we used psycopg2 with parameterised SQL queries instead of an ORM. On the frontend, we used vanilla JavaScript, Chart.js to generate six charts, and Leaflet.js to display taxi activity across NYC taxi zones. This meant we did not need an additional frontend framework or bundler. The database has a basic star schema with taxi_zones as the dimension table and trips as the fact table. All 265 taxi zones are stored in taxi_zones, including their IDs, names, and boroughs. The trips table contains the trip records and derived features. The foreign keys trips.pulocationid and trips.dolocationid reference taxi_zones.location_id. This enables trips to be grouped by pickup borough and other attributes. We created six indexes, including one on (pickup_hour, pickup_dow) for the speed heatmap query. There were several trade-offs in our design decisions. Since our source data was a single file containing one month of trips, we chose batch processing instead of streaming. The endpoints for all six charts were called in parallel rather than one by one, allowing the dashboard visuals to load faster. However, this meant multiple database queries could run simultaneously, which should be considered in a production application.

An alternative to our custom min-heap in algorithms/top_k.py would be SQL ORDER BY and LIMIT, pandas sort_values(), Python's built-in sorted(), or the Counter class. We implemented our own approach not to improve performance at this scale, but to demonstrate our understanding of data structures and algorithms.

3. Algorithms and Data Structures

Algorithm 1: Custom Min-Heap for Top-K Zones

To find the busiest zones based on the number of trips, we used a custom min-heap in backend/algorithms/top_k.py. The algorithm processes 265 zones and finds the top K without using heapq, sorted, sort_values, or Counter.
The min-heap maintains up to K elements. If it contains fewer than K elements, we add the next element. Once it is full, we compare the new value with the smallest value at the root. If the new value is larger, we remove the root and add the new element. The algorithm looks like this:

```
function top_k(data, k):
   heap = []

   for each (zone_id, value) in data:
      if len(heap) < k:
         heap_push(heap, (value, zone_id))
      else if value > heap[0].value:
         heap_pop(heap)
         heap_push(heap, (value, zone_id))

   return the top K elements in descending order
```

The heap_push and heap_pop operations take (O(\log K)) time. The overall time complexity is (O(N \log K)), and the space complexity is (O(K)).
For our dataset of 265 zones and K = 10, the heap allows us to find the top ten zones without sorting every zone. A full sort takes (O(N \log N)), while the heap takes (O(N \log K)). The difference is not very significant at this scale, but the heap shows how an appropriate data structure can provide an efficient solution to the top-K problem.

Algorithm 2: Custom Insertion Sort
In frontend/assets/js/sort.js, we implemented insertion sort to sort up to 500 records by the selected numeric column without using JavaScript's built-in Array.sort() method.
Insertion sort compares the second element with the first and moves larger elements to the right. It then compares the third element with the elements before it, moving larger elements until the third element reaches the correct position. This process continues until the array is sorted.
The average and worst-case time complexity is (O(n^2)), and the extra space requirement is (O(1)) when implemented in place. The algorithm is easy to implement and works well with small datasets, which suits our case.

4. Insights and Interpretation

Insight 1: Rush-Hour Fare Differences
We found that average fares were higher for trips starting between 07:00–09:00 and 16:00–19:00, approximately 18–24% above overnight baseline fares, even though the trips had comparable distances.
We also looked at fare_per_mile to check whether distance was the only factor behind the difference in fares during rush hour. The findings suggest that it was not the only factor. Congestion, trip duration, and trip type may also contribute to higher fares during these periods.
This information helps us understand cost differences throughout the day. However, more research is needed to separate the effects of congestion, trip duration, tolls, and other factors before drawing firm conclusions.
Insight 2: Manhattan Has the Most Pickups, While Queens Has a Higher Fare per Mile
Our analysis found that Manhattan accounted for approximately 72% of all pickups in our dataset, making it the largest pickup area in our sample. However, the average fare per mile for trips

associated with Queens was higher than for Manhattan, at approximately $4.20 compared with $3.10 per mile.
One possible explanation is that airport trips involving JFK and LaGuardia may have different fare structures, tolls, and distances compared with shorter urban trips. However, further analysis is needed to confirm how much these trips contribute to the difference.
Separating airport trips from other journeys would help us understand their contribution to the average fare-per-mile difference.

Insight 3: Lower Speeds on Saturday Night

During our analysis, we found that average trip speeds were particularly low between Saturday at 22:00 and Sunday at 02:00, reaching approximately 6.8 mph. This was lower than the Friday rush-hour average of 9.2 mph.
Trip activity remained high even though average speeds were low. This suggests that reduced demand alone may not explain the lower speeds. Nightlife traffic and congestion may be contributing factors, but further research is needed to identify the main causes.
These findings may be useful for studying late-night traffic patterns and identifying opportunities to improve travel times.

5. Reflection and Future Work

Technical Challenges

We encountered several technical problems during development.

- When migrating from SQLite to PostgreSQL, we found SQL compatibility issues, including the use of a reserved word as a JOIN alias and the need to cast double precision values to numeric when using ROUND(). These issues highlighted the importance of testing with the intended database early in development.
- We also experienced a Python version conflict. A pre-release version of Python caused a SystemError: unknown slot ID when working with compiled pandas extensions. We resolved this by using a stable Python version in our virtual environment.
- Lastly, the map did not render correctly when its parent view was hidden. Leaflet could not calculate the map container's size, so the map tiles did not appear as expected. We fixed this by calling invalidateSize() when the map became visible.

Team Challenges

To prevent integration problems between the backend and frontend, we agreed on the API parameter names (hour, dow, and pu_borough) early in development. We also fixed a .gitignore merge conflict. This taught us the importance of communicating before editing shared files.

Future Work

There are two main areas we would like to improve in this project.
The first is real-time data processing. Our current ETL pipeline processes a fixed monthly dataset. If live trip data became available, a streaming solution such as Apache Kafka could help us process it continuously.
The second is predictive analytics. The patterns we discovered in our hourly and borough-based analysis could be used to develop a model that predicts demand and presents its predictions directly on the dashboard. This would extend the project beyond descriptive analytics and allow users to explore possible future demand patterns.